

Санкт-Петербургский политехнический университет Петра  
Великого

Институт компьютерных наук и кибербезопасности  
Высшая школа программной инженерии

**Приоритизация тестовых случаев на основе генетического  
алгоритма**

по дисциплине «Методы тестирования программного обеспечения»

**Выполнил**

студент гр.5130903/40003

Я. М. Качанов

**Руководитель**

старший преподаватель

ВШПИ

В. А. Пархоменко

Санкт-Петербург – 2026

## СОДЕРЖАНИЕ

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Введение .....                                                        | 3  |
| 1 Анализ существующих методов .....                                   | 4  |
| 1.1 Регрессионное тестирование и приоритизация тестовых случаев ..... | 4  |
| 1.2 Существующие методы приоритизации .....                           | 4  |
| 2 Постановка задачи .....                                             | 5  |
| 2.1 Формальное определение .....                                      | 5  |
| 2.2 Метрики оценки эффективности .....                                | 6  |
| 3 Предлагаемый метод .....                                            | 6  |
| 3.1 Фитнесс-функция .....                                             | 6  |
| 3.2 Операторы генетического алгоритма .....                           | 7  |
| 3.3 Псевдокод алгоритма .....                                         | 9  |
| 4 Иллюстративный пример .....                                         | 10 |
| 4.1 Исходные данные .....                                             | 10 |
| 4.2 Шаг 1. Инициализация .....                                        | 10 |
| 4.3 Шаг 2. Вычисление фитнеса .....                                   | 10 |
| 4.4 Шаг 3. Кроссовер ОХ .....                                         | 12 |
| 4.5 Шаг 4. Результат .....                                            | 12 |
| 5 Программная реализация .....                                        | 12 |
| 5.1 Структура проекта .....                                           | 13 |
| 5.2 Описание ключевых классов .....                                   | 13 |
| 5.3 Форматы входных данных .....                                      | 14 |
| 5.4 Инструкция по установке и запуску .....                           | 15 |
| 6 Экспериментальная оценка .....                                      | 16 |
| 6.1 Наборы данных и параметры эксперимента .....                      | 16 |
| 6.2 Анализ чувствительности весов .....                               | 17 |
| 6.3 Результаты на BigFaultMatrix .....                                | 18 |
| 6.4 Результаты на Defects4J .....                                     | 19 |
| 6.5 Анализ результатов .....                                          | 20 |
| Заключение .....                                                      | 21 |
| Список использованных источников .....                                | 22 |
| Приложение 1 Программный код .....                                    | 23 |

## ВВЕДЕНИЕ

**Актуальность.** Регрессионное тестирование является неотъемлемой частью процесса разработки программного обеспечения: после каждого изменения кода необходимо убедиться, что существующая функциональность не нарушена. По мере роста проекта тестовый набор увеличивается экспоненциально, что делает полный повторный прогон тестов дорогостоящим и времязатратным. Приоритизация тестовых случаев (Test Case Prioritization, TCP) позволяет упорядочить выполнение тестов таким образом, чтобы дефекты обнаруживались как можно раньше, не исключая ни одного теста из набора. Применение эволюционных методов оптимизации, в частности генетических алгоритмов, открывает возможность учитывать одновременно несколько критериев качества при формировании порядка выполнения тестов.

**Предмет исследования.** Методы приоритизации тестовых случаев на основе генетического алгоритма с многокомпонентной фитнес-функцией, включающей покрытие кода, оценку вероятности дефектов (fault-proneness) и структурное разнообразие тестов (diversification) на основе расстояния Жаккара.

**Цель исследования.** Разработка и экспериментальная оценка алгоритма приоритизации тестовых случаев, превосходящего базовый генетический алгоритм и жадный алгоритм дополнительного покрытия по метрикам APFD и APFDc на реальных наборах данных.

### **Задачи:**

1. Изучить существующие методы приоритизации тестовых случаев и метрики оценки их эффективности.
2. Разработать фитнес-функцию генетического алгоритма, учитывающую покрытие кода, fault-proneness и структурное разнообразие тестов.
3. Реализовать алгоритм на языке Java с поддержкой загрузки данных в форматах CSV и JSON.
4. Провести сравнительный эксперимент на разных наборах данных.
5. Проанализировать результаты и определить условия применимости предложенного подхода.

## 1 АНАЛИЗ СУЩЕСТВУЮЩИХ МЕТОДОВ

В данной главе рассматриваются основные подходы к приоритизации тестовых случаев, их преимущества и ограничения. В параграфе 1.1 описана задача регрессионного тестирования и приоритизации тестовых случаев. В параграфе 1.2 рассмотрены существующие методы приоритизации и их ограничения.

### 1.1 Регрессионное тестирование и приоритизация тестовых случаев

Регрессионное тестирование — процесс повторного выполнения тестов после внесения изменений в программный код с целью подтверждения того, что ранее работавшая функциональность не нарушена. По мере развития проекта объём тестового набора растёт, что делает полный повторный прогон нецелесообразным с точки зрения временных и вычислительных затрат.

Для решения данной проблемы используются три основных подхода: минимизация тестового набора (Test Suite Minimization), отбор тестов (Test Case Selection) и приоритизация тестовых случаев (Test Case Prioritization, TCP). В отличие от первых двух, TCP не исключает тесты из набора, а только упорядочивает их выполнение по некоторому критерию, сохраняя полноту покрытия.

### 1.2 Существующие методы приоритизации

Среди методов приоритизации выделяют несколько классов, каждый из которых имеет характерные ограничения.

Случайный поиск (Random Search) упорядочивает тесты в произвольном порядке без учёта каких-либо критериев. Несмотря на простоту, он служит нижней границей сравнения — любой разумный алгоритм приоритизации должен превосходить случайный порядок.

Жадные алгоритмы на основе покрытия кода упорядочивают тесты по убыванию числа новых покрываемых элементов. Алгоритм дополнительного покрытия (Additional) на каждом шаге выбирает тест с максимальным числом ещё не покрытых элементов — это простой и эффективный подход, однако он не учитывает вероятность дефектов и принимает локально оптимальные решения, не гарантируя глобального оптимума.

Эволюционные методы, в частности генетические алгоритмы, рассматривают задачу ТСП как задачу оптимизации перестановок. В работе [0] рассматривается применение ГА к набору данных BigFaultMatrix, показано превосходство над случайным поиском и алгоритмами hill climbing. Вместе с тем фитнес-функция учитывает только скорость обнаружения дефектов, не принимая во внимание вероятность дефектов и структурное разнообразие тестов.

В статье [0] рассматривается подход CovClustering, в котором вероятность дефектов (fault-proneness) методов оценивается на основе анализа влияния изменений (Change Impact Analysis, CIA). Подход учитывает вероятность дефектов, однако использует кластеризацию вместо эволюционного поиска, что ограничивает возможности глобальной оптимизации порядка тестов.

Таким образом, ни один из рассмотренных методов не сочетает одновременно эволюционный поиск, учёт вероятности дефектов и структурное разнообразие тестов. Устранение этих ограничений является целью настоящей работы.

## 2 ПОСТАНОВКА ЗАДАЧИ

В данной главе приведена формальная постановка задачи приоритизации тестовых случаев и определены метрики оценки эффективности. В параграфе 2.1 дано формальное определение задачи. В параграфе 2.2 описаны метрики APFD и APFDc.

### 2.1 Формальное определение

Пусть  $T = \{t_1, t_2, \dots, t_n\}$  — тестовый набор из  $n$  тестовых случаев. Каждый тестовый случай  $t_i$  характеризуется множеством покрываемых методов  $M_i \subseteq M$ , вероятностью дефектов  $r_i \in [0, 1]$  и временем выполнения  $c_i > 0$ .

Задача приоритизации состоит в нахождении перестановки  $T^*$ :

$$T^* = \arg \max_{T' \in \Pi(T)} f(T')$$

где  $\Pi(T)$  — множество всех перестановок  $T$ , целевая функция:

$$f(T') = w_c \cdot \text{Cov}(T') + w_r \cdot \text{FP}(T') + w_d \cdot \text{Div}(T'), \quad w_c + w_r + w_d = 1$$

## 2.2 Метрики оценки эффективности

Основной метрикой оценки является APFD (Average Percentage of Faults Detected) [0]:

$$APFD = 1 - \frac{\sum_{i=1}^m TF_i}{n \cdot m} + \frac{1}{2n} \quad (2.1)$$

где  $n$  — количество тестов,  $m$  — количество дефектов,  $TF_i$  — позиция теста, первым обнаружившего  $i$ -й дефект. Значение APFD близкое к 1 означает, что дефекты обнаруживаются в начале выполнения набора.

Метрика APFDc расширяет APFD с учётом стоимости выполнения каждого теста:

$$APFDc = \frac{\sum_{i=1}^m \left( t_{TF_i} - \frac{c_{TF_i}}{2} \right)}{T_{total} \cdot m}$$

где  $t_{TF_i}$  — суммарное время выполнения тестов до момента обнаружения  $i$ -го дефекта,  $c_{TF_i}$  — время выполнения теста обнаружившего дефект,  $T_{total}$  — суммарное время выполнения всего набора.

## 3 ПРЕДЛАГАЕМЫЙ МЕТОД

В данной главе описан разработанный алгоритм приоритизации тестовых случаев GA-FP+Div, представляющий собой генетический алгоритм с улучшенной многокомпонентной фитнес-функцией.

### 3.1 Фитнесс-функция

Фитнесс-функция оценивает качество предложенной последовательности тестов по трём нормализованным компонентам. Использование нескольких компонент позволяет учитывать не только покрытие кода, но и рискованность покрываемых методов и структурное разнообразие тестов.

**Покрывание кода (Cov)** — доля уникальных методов, покрытых последовательностью тестов, от общего числа методов проекта:

$$Cov(T') = \frac{|\bigcup_{i=1}^n M_i|}{|M|}$$

Числитель — количество уникальных методов, покрытых хотя бы одним тестом из последовательности. Знаменатель — общее число методов в проекте. Результат нормализован к диапазону  $[0, 1]$ .

**Вероятность дефектов (FP)** — средняя вероятность дефектов первых  $k$  тестов последовательности:

$$FP(T') = \frac{1}{k} \sum_{i=1}^k r_{\pi(i)}$$

где  $r_{\pi(i)}$  — вероятность дефектов  $i$ -го теста в последовательности, оцениваемая на основе анализа влияния изменений CIA (Change Impact Analysis). Компонента максимизирует вероятность того, что наиболее рискованные тесты выполняются первыми.

**Структурное разнообразие (Div)** — среднее расстояние Жаккара между соседними тестами в первых  $k$  позициях:

$$Div(T') = \frac{1}{k-1} \sum_{i=1}^{k-1} d_J(M_{\pi(i)}, M_{\pi(i+1)})$$

где расстояние Жаккара определяется как:

$$d_J(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$$

Значение  $d_J = 0$  означает что тесты покрывают одинаковые методы,  $d_J = 1$  — покрытия не пересекаются. Компонента стимулирует алгоритм чередовать тесты с различным покрытием, снижая дублирование в начале последовательности.

Все три компоненты нормализованы к диапазону  $[0, 1]$ . Итоговый фитнесс вычисляется как взвешенная сумма:

$$f(T') = w_c \cdot Cov(T') + w_r \cdot FP(T') + w_d \cdot Div(T')$$

где  $w_c + w_r + w_d = 1$ ,  $w_c, w_r, w_d \geq 0$ . В настоящей работе приняты значения  $w_c = 0,2$ ,  $w_r = 0,6$ ,  $w_d = 0,2$ ,  $k = 10$ .

### 3.2 Операторы генетического алгоритма

Генетический алгоритм оперирует популяцией из  $N$  особей, каждая из которых представляет собой перестановку тестового набора  $T$ . На каждом поколении популяция обновляется с помощью операторов селекции, кроссовера и мутации.

**Инициализация.** Начальная популяция из  $N$  особей формируется случайным перемешиванием тестового набора  $T$ . Использование фиксированного зерна генератора случайных чисел обеспечивает воспроизводимость результатов.

**Селекция.** Используется турнирная селекция с размером турнира  $s = 3$ : случайно выбираются  $s$  особей, из которых в следующее поколение передаётся особь с наибольшим значением фитнеса. Турнирная селекция обеспечивает селективное давление без полного исключения слабых особей.

**Кроссовер.** Применяется оператор ОХ (Order Crossover), сохраняющий валидность перестановки. Алгоритм ОХ работает следующим образом:

- А. Случайно выбирается сегмент  $[\alpha, \beta]$  из первого родителя  $P_1$  и копируется в потомка  $C$  на те же позиции.
- В. Оставшиеся позиции  $C$  заполняются элементами из второго родителя  $P_2$  в порядке их появления, пропуская элементы уже присутствующие в сегменте.

Например, для  $P_1 = [t_3, t_1, t_5, t_2, t_4]$  и  $P_2 = [t_2, t_4, t_1, t_5, t_3]$  при сегменте  $[1, 3]$ :

$$C = [t_2, \underbrace{t_1, t_5, t_2}_{\text{сегмент из } P_1}, t_3] \rightarrow [t_4, t_1, t_5, t_2, t_3]$$

ОХ гарантирует что потомок является корректной перестановкой без дубликатов, что критично для задачи ТСР.

**Мутация.** Выполняется  $\lfloor \sqrt{n} \rfloor$  случайных перестановок пар элементов (swar), где  $n$  — размер тестового набора. Адаптивная интенсивность мутации позволяет сохранять разнообразие популяции при различных размерах тестовых наборов: для  $n = 100$  выполняется 10 перестановок, для  $n = 1000$  — 31.

**Элитизм.** Лучшая особь текущего поколения переносится в следующее поколение без изменений. Это гарантирует монотонное неубывание лучшего найденного решения на протяжении всей эволюции.

**Параметры алгоритма.** Вероятность кроссовера  $p_c = 0,8$  и вероятность мутации  $p_m = 0,2$  выбраны в соответствии с рекомендациями [0]. Размер популяции  $N = 100$ . Число поколений задаётся адаптивно:  $G = \max(50, \lfloor \log_2(n) \rfloor \cdot 10)$ .

### 3.3 Псевдокод алгоритма

Алгоритм работает следующим образом. На этапе инициализации формируется популяция  $P$  из  $N$  случайных перестановок тестового набора  $T$ , из которых выбирается лучшая особь  $T^*$ .

В каждом из  $G$  поколений формируется новая популяция  $P'$ . Первой в неё помещается текущая лучшая особь  $T^*$  без изменений (элитизм). Затем в цикле до заполнения популяции выполняются следующие шаги: турнирной селекцией выбираются два родителя  $P_1$  и  $P_2$ ; с вероятностью  $p_c$  к ним применяется ОХ-кроссовер, порождающий двух потомков  $C_1$  и  $C_2$ ; с вероятностью  $p_m$  каждый потомок подвергается мутации. Сформированные потомки добавляются в  $P'$ .

После обновления популяции проверяется улучшение: если лучшая особь нового поколения превосходит  $T^*$  по значению фитнеса,  $T^*$  обновляется. По завершении всех поколений алгоритм возвращает  $T^*$  — наилучшую найденную перестановку тестового набора.

Листинг 3.1

#### Псевдокод алгоритма GA-FP+Div

```

Вход:  $T$ ,  $N$ ,  $G$ ,  $p_c$ ,  $p_m$ ,  $w_c$ ,  $w_r$ ,  $w_d$ ,  $k$ 
Выход:  $T^*$  -- оптимальная перестановка

5  $P \leftarrow \text{InitPopulation}(T, N)$ 
 $T^* \leftarrow \text{argmax } f(P)$ 

для  $g$  от 1 до  $G$ :
     $P' \leftarrow \{T^*\}$                                      // элитизм
10 пока  $|P'| < N$ :
         $P_1 \leftarrow \text{Tournament}(P, 3)$ 
         $P_2 \leftarrow \text{Tournament}(P, 3)$ 
        если  $\text{rand}() < p_c$ :
             $C_1, C_2 \leftarrow \text{OXCrossover}(P_1, P_2)$ 
15 иначе:
             $C_1 \leftarrow P_1$ ;  $C_2 \leftarrow P_2$ 
        если  $\text{rand}() < p_m$ :
             $\text{Mutate}(C_1, \text{floor}(\text{sqrt}(n)))$ 
        если  $\text{rand}() < p_m$ :
20          $\text{Mutate}(C_2, \text{floor}(\text{sqrt}(n)))$ 
         $P' \leftarrow P' + \{C_1, C_2\}$ 
     $P \leftarrow P'$ 
    если  $f(\text{best}(P)) > f(T^*)$ :
         $T^* \leftarrow \text{best}(P)$ 

```

4 ИЛЛЮСТРАТИВНЫЙ ПРИМЕР

В данной главе приведено пошаговое выполнение алгоритма GA-FP+Div на небольшом примере. Рассматривается тестовый набор из 5 тестовых случаев и 4 дефектов.

4.1 Исходные данные

Пусть дан тестовый набор  $T = \{t_1, t_2, t_3, t_4, t_5\}$  и множество дефектов  $F = \{f_1, f_2, f_3, f_4\}$ . Матрица покрытия дефектов и значения вероятности дефектов приведены в таблице 4.1.

Таблица 4.1

Исходные данные иллюстративного примера

| Тест  | $f_1$ | $f_2$ | $f_3$ | $f_4$ | $r_i$ |
|-------|-------|-------|-------|-------|-------|
| $t_1$ | 1     | 0     | 1     | 0     | 0,3   |
| $t_2$ | 0     | 1     | 0     | 1     | 0,6   |
| $t_3$ | 1     | 1     | 0     | 0     | 0,4   |
| $t_4$ | 0     | 0     | 1     | 1     | 0,7   |
| $t_5$ | 1     | 0     | 0     | 1     | 0,5   |

Параметры алгоритма:  $N = 2$  особи,  $G = 1$  поколение,  $p_c = 0,8$ ,  $p_m = 0,2$ ,  $w_c = 0,4$ ,  $w_r = 0,4$ ,  $w_d = 0,2$ ,  $k = 3$ .

4.2 Шаг 1. Инициализация

Начальная популяция формируется случайным перемешиванием:

$$P_1 = [t_3, t_1, t_5, t_2, t_4], \quad P_2 = [t_2, t_4, t_1, t_5, t_3]$$

4.3 Шаг 2. Вычисление фитнесса

Вычислим фитнесс для  $P_1 = [t_3, t_1, t_5, t_2, t_4]$  при  $k = 3$ .

**Покрытие:** объединение покрытий всех тестов:

$$M_3 \cup M_1 \cup M_5 \cup M_2 \cup M_4 = \{f_1, f_2, f_3, f_4\}$$

$$\text{Cov}(P_1) = \frac{4}{4} = 1,0$$

**Вероятность дефектов** первых  $k = 3$  тестов  $(t_3, t_1, t_5)$ :

$$\text{FP}(P_1) = \frac{r_3 + r_1 + r_5}{3} = \frac{0,4 + 0,3 + 0,5}{3} = 0,400$$

**Структурное разнообразие** между соседними парами в топ-3:

Расстояние Жаккара между  $t_3$  и  $t_1$ :

$$d_J(M_3, M_1) = 1 - \frac{|\{f_1\}|}{|\{f_1, f_2, f_3\}|} = 1 - \frac{1}{3} = 0,667$$

Расстояние Жаккара между  $t_1$  и  $t_5$ :

$$d_J(M_1, M_5) = 1 - \frac{|\{f_1\}|}{|\{f_1, f_3, f_4\}|} = 1 - \frac{1}{3} = 0,667$$

$$\text{Div}(P_1) = \frac{0,667 + 0,667}{2} = 0,667$$

**Итоговый фитнесс:**

$$f(P_1) = 0,4 \cdot 1,0 + 0,4 \cdot 0,400 + 0,2 \cdot 0,667 = 0,400 + 0,160 + 0,133 = 0,693$$

Аналогично для  $P_2 = [t_2, t_4, t_1, t_5, t_3]$ :

$$\text{FP}(P_2) = \frac{0,6 + 0,7 + 0,3}{3} = 0,533$$

$$d_J(M_2, M_4) = 1 - \frac{0}{4} = 1,000, \quad d_J(M_4, M_1) = 1 - \frac{1}{4} = 0,750$$

$$\text{Div}(P_2) = \frac{1,000 + 0,750}{2} = 0,875$$

$$f(P_2) = 0,4 \cdot 1,0 + 0,4 \cdot 0,533 + 0,2 \cdot 0,875 = 0,400 + 0,213 + 0,175 = 0,788$$

Таким образом,  $f(P_2) > f(P_1)$ , поэтому  $T^* = P_2$ .

#### 4.4 Шаг 3. Кроссовер ОХ

Турнирная селекция выбирает  $P_1$  и  $P_2$  как родителей. Применяем ОХ-кроссовер с сегментом  $[1, 3]$ :

$$P_1 = [t_3, \underbrace{t_1, t_5, t_2}_{\text{сегмент}}, t_4]$$

$$P_2 = [t_2, t_4, t_1, t_5, t_3]$$

Потомок  $C_1$ : копируем сегмент  $[t_1, t_5, t_2]$  из  $P_1$ , остаток заполняем из  $P_2$ , пропуская  $t_1, t_5, t_2$ :

$$C_1 = [t_4, t_1, t_5, t_2, t_3]$$

#### 4.5 Шаг 4. Результат

Вычислим фитнес потомка  $C_1 = [t_4, t_1, t_5, t_2, t_3]$ :

$$FP(C_1) = \frac{0,7 + 0,3 + 0,5}{3} = 0,500$$

$$d_J(M_4, M_1) = 0,750, \quad d_J(M_1, M_5) = 0,667$$

$$Div(C_1) = \frac{0,750 + 0,667}{2} = 0,708$$

$$f(C_1) = 0,4 \cdot 1,0 + 0,4 \cdot 0,500 + 0,2 \cdot 0,708 = 0,400 + 0,200 + 0,142 = 0,742$$

Поскольку  $f(C_1) = 0,742 < f(T^*) = 0,788$ , лучшая особь не обновляется. Алгоритм возвращает:

$$T^* = [t_2, t_4, t_1, t_5, t_3]$$

Данная последовательность ставит вперёд тесты с высокой вероятностью дефектов ( $t_2: r = 0,6, t_4: r = 0,7$ ) и разнообразным покрытием, что соответствует цели алгоритма.

### 5 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

В данной главе описана программная реализация алгоритма GA-FP+Div. В параграфе 5.1 описана структура проекта. В параграфе 5.2 описаны ключевые классы. В параграфе 5.3 описаны форматы входных данных. В параграфе 5.4 приведена инструкция по установке и запуску.

## 5.1 Структура проекта

Программная реализация выполнена на языке Java с использованием системы сборки Maven. Проект организован в следующие пакеты:

- `com.tcp.algorithm` — алгоритмы приоритизации: `AdditionalPrioritizer`, `GeneticPrioritizer`, `RandomPrioritizer`, интерфейс `Prioritizer`;
- `com.tcp.fitness` — фитнес-функция: `GaFitnessFunction`;
- `com.tcp.data` — загрузка данных: `DataLoader`;
- `com.tcp.metric` — метрики оценки: `ApfdCalculator`, `ApfdCostCalculator`;
- `com.tcp.model` — модель данных: `TestCase`;
- `com.tcp` — точка входа: `Main`.

## 5.2 Описание ключевых классов

**Интерфейс** `Prioritizer` определяет единый программный интерфейс для всех алгоритмов приоритизации: метод `prioritize(List<TestCase>)` принимает неупорядоченный пул тестов и возвращает упорядоченный список.

**Класс** `TestCase` представляет тестовый случай и содержит четыре поля: идентификатор теста, множество покрываемых методов, вероятность дефектов и время выполнения.

**Класс** `GaFitnessFunction` реализует многокомпонентную фитнес-функцию. Метод `calculateFitness(List<TestCase>)` вычисляет взвешенную сумму трёх компонент: покрытия, вероятности дефектов и структурного разнообразия. Веса компонент задаются в конструкторе.

**Класс** `GeneticPrioritizer` реализует генетический алгоритм. Конструктор принимает размер популяции, число поколений, вероятности кроссовера и мутации, а также экземпляр `GaFitnessFunction`. Метод `prioritize` выполняет эволюционный цикл и возвращает лучшую найденную перестановку.

**Класс** `DataLoader` обеспечивает загрузку данных трёх форматов: `BigFaultMatrix`, `Defects4J` и синтетических данных. Все форматы приводятся к единому внутреннему представлению `FaultMatrixData`, содержащему список тестов, карту убитых мутантов и общее число мутантов.

**Класс** `ApfdCalculator` реализует расчёт метрики APFD по формуле (2.1).

**Класс** `ApfdCostCalculator` реализует расчёт метрики APFDc с учётом времени выполнения тестов.

### 5.3 Форматы входных данных

Программа поддерживает три формата входных данных.

**BigFaultMatrix** — бинарная матрица тест × дефект в текстовом формате. Каждая строка соответствует одному тесту: первый элемент — идентификатор теста, остальные — значения 0 или 1, где 1 означает что тест обнаруживает соответствующий дефект.

Листинг 5.1

#### Фрагмент BigFaultMatrix

```
t1,0,1,0,1,0,0,1,...
t2,1,0,1,0,1,0,0,...
```

**Defects4J** — данные реальных Java-проектов. Для каждого проекта используются три файла:

- coverage/matrix.csv — матрица покрытия методов (строки: тесты, столбцы: методы);
- mutants/kill.csv, testMap.csv, covMap.csv — информация об убитых мутантах;
- cia/impact\_probabilities.csv — вероятности дефектов методов на основе CIA-анализа.

**Синтетические данные** генерируются программно методом

`DataLoader.generateSynthetic(numTests, numFaults, density)`, принимающим три параметра: количество тестов, количество дефектов и плотность покрытия. Для обеспечения нетривиального сравнения алгоритмов тесты делятся на три группы:

- Тесты с широким покрытием и низким риском (30% тестов) — тесты с широким покрытием обычных дефектов, но низкой вероятностью дефектов ( $r_i \in [0,05; 0,10]$ ). Жадный алгоритм склонен выбирать их первыми из-за большого покрытия, однако они не покрывают наиболее рискованные методы.
- Тесты с узким покрытием и высоким риском (20% тестов) — тесты с узким покрытием, но высокой вероятностью дефектов ( $r_i \in [0,60; 1,00]$ ). Покрывают первые 40% дефектов, считающихся рискованными. Алгоритм GA-FP+Div находит их раньше за счёт компоненты вероятности дефектов.

- Обычные тесты (50% тестов) — тесты со случайным покрытием с заданной плотностью `density`. Вероятность дефектов пропорциональна доле покрытых дефектов.

Такая структура позволяет наглядно продемонстрировать различие в стратегиях алгоритмов: `Additional` выбирает тесты с максимальным покрытием методов, тогда как `GA-FP+Div` ставит в приоритет тесты с высокой вероятностью дефектов.

Результаты экспериментов сохраняются в двух форматах: `results/results.csv` для анализа в табличных редакторах и `results/results.json` для программной обработки.

## 5.4 Инструкция по установке и запуску

**Требования:** Java 11 или выше, Apache Maven 3.6 или выше.

**Сборка проекта:**

```
| mvn clean package
```

**Запуск:**

```
| java -jar target/tcp-1.0.jar
```

После запуска отображается интерактивное меню выбора режима работы:

```
| === TCP Benchmark ===
| 1. BigFaultMatrix (4 вариации)
| 2. Defects4J
5 | 3. Синтетические данные
| 4. Все Defects4J проекты
| 0. Выход
```

Для запуска на данных `Defects4J` необходимо указать путь к папке проекта, содержащей подпапки `coverage`, `mutants` и `cia`. Результаты автоматически сохраняются в папку `results/` при выходе из программы.

**Эталонный прогон.** Для проверки корректности работы программы предусмотрен запуск на наборе данных `BigFaultMatrix` (пункт меню 1). Ожидаемые результаты для оригинальной матрицы:

Эталонные результаты на BigFaultMatrix

| Алгоритм   | APFD   | APFDc  |
|------------|--------|--------|
| Random     | 0,9230 | 0,6861 |
| Additional | 0,9987 | 0,7617 |
| GA-Base    | 0,9168 | 0,6799 |
| GA-FP+Div  | 0,9590 | 0,7221 |

## 6 ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА

В данной главе представлены результаты сравнительного эксперимента. В параграфе 6.1 описаны наборы данных и параметры эксперимента. В параграфе 6.3 приведены результаты на наборе данных BigFaultMatrix. В параграфе 6.4 приведены результаты на проектах Defects4J. В параграфе 6.5 выполнен анализ результатов.

### 6.1 Наборы данных и параметры эксперимента

В эксперименте используются два публичных набора данных.

**BigFaultMatrix** [0] — бинарная матрица размером  $1000 \times 38$ , где строки соответствуют тестовым случаям, столбцы — дефектам. Для расширения бенчмарка дополнительно генерируются три вариации путем случайного инвертирования 10%, 20% и 30% ячеек матрицы.

**Defects4J** [0] — репозиторий реальных дефектов в Java-проектах с открытым исходным кодом. Используются данные репозитория предоставляющего матрицы покрытия методов, карты убитых мутантов и оценки вероятности дефектов на основе CIA-анализа.

В эксперименте задействованы 7 проектов, характеристики которых приведены в таблице 6.1.

Характеристики проектов Defects4J

| Проект              | Тестов | Мутантов |
|---------------------|--------|----------|
| Jsoup/93            | 690    | 30       |
| Csv/16              | 284    | 99       |
| JacksonDatabind/112 | 36     | 37       |
| Time/27             | 3768   | 828      |
| Codec/18            | 834    | 26       |
| JacksonCore/26      | 108    | 2002     |
| Lang/65             | 1609   | 351      |

Сравниваются четыре алгоритма:

- **Random** — случайная перестановка тестов, нижняя граница сравнения. Результат усредняется по 5 прогонам.
- **Additional** — жадный алгоритм дополнительного покрытия, классический baseline для TCP.
- **GA-Base** — генетический алгоритм с фитнес-функцией только по покрытию ( $w_c = 1,0$ ,  $w_r = 0$ ,  $w_d = 0$ ).
- **GA-FP+Div** — предлагаемый метод с фитнес-функцией по покрытию, вероятности дефектов и структурному разнообразию ( $w_c = 0,2$ ,  $w_r = 0,6$ ,  $w_d = 0,2$ ).

Параметры генетических алгоритмов: размер популяции  $N = 100$ , вероятность кроссовера  $p_c = 0,8$ , вероятность мутации  $p_m = 0,2$ , число поколений

$$G = \max(50, \lfloor \log_2(n) \rfloor \cdot 10)$$

. Эксперименты проводились на фиксированном зерне генератора случайных чисел для воспроизводимости результатов.

## 6.2 Анализ чувствительности весов

Для обоснования выбора весов фитнес-функции проведён анализ чувствительности: перебраны все комбинации  $w_c$ ,  $w_r$ ,  $w_d$  с шагом 0,2 при условии  $w_c + w_r + w_d = 1$ . Для каждой конфигурации вычислялся средний APFD по всем 7 проектам Defects4J.

Результаты показали что алгоритм устойчив к выбору весов: конфигурации с  $w_r \geq 0,2$  дают средний APFD в диапазоне  $[0,96; 0,98]$ . Конфигурации без ком-

поненты вероятности дефектов ( $w_r = 0$ ) значительно уступают — средний APFD составляет 0,929, что подтверждает вклад CIA-данных в качество приоритизации.

Оптимальная конфигурация  $w_c = 0,2$ ,  $w_r = 0,6$ ,  $w_d = 0,2$  обеспечивает средний APFD = 0,9785 и принята в настоящей работе как основная.

### 6.3 Результаты на BigFaultMatrix

Результаты эксперимента на четырёх вариациях BigFaultMatrix приведены в таблице 6.2 и на рисунке 6.1.

Таблица 6.2

Результаты на BigFaultMatrix (APFD)

| Вариация | Random | Additional    | GA-Base | GA-FP+Div |
|----------|--------|---------------|---------|-----------|
| Оригинал | 0,9230 | <b>0,9987</b> | 0,9168  | 0,9590    |
| Flip 10% | 0,9925 | <b>0,9983</b> | 0,9914  | 0,9973    |
| Flip 20% | 0,9962 | <b>0,9988</b> | 0,9962  | 0,9984    |
| Flip 30% | 0,9977 | <b>0,9989</b> | 0,9967  | 0,9987    |

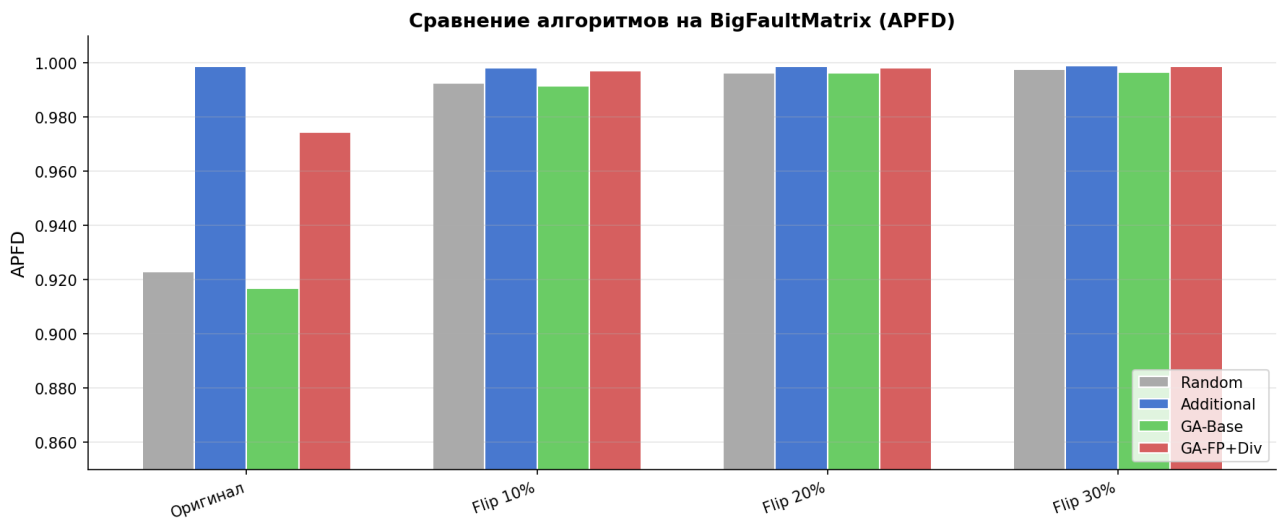


Рис.6.1. Сравнение алгоритмов на BigFaultMatrix по метрике APFD

На данном наборе данных алгоритм Additional показывает наилучшие результаты во всех четырёх вариациях. Это объясняется структурой BigFaultMatrix: покрытие тестов непосредственно соответствует обнаружению дефектов, что делает жадную стратегию близкой к оптимальной. Генетические алгоритмы уступают Additional, поскольку сигнал вероятности дефектов в данном формате слаб.

6.4 Результаты на Defects4J

Результаты эксперимента на проектах Defects4J приведены в таблице 6.3 и на рисунке 6.2.

Таблица 6.3

Результаты на проектах Defects4J (APFD)

| Проект              | Random | Additional    | GA-Base | GA-FP+Div     |
|---------------------|--------|---------------|---------|---------------|
| Jsoup/93            | 0,9115 | 0,9811        | 0,9936  | <b>0,9993</b> |
| Csv/16              | 0,9915 | 0,9958        | 0,9925  | <b>0,9982</b> |
| JacksonDatabind/112 | 0,8945 | 0,8803        | 0,8697  | <b>0,9163</b> |
| Time/27             | 0,9876 | <b>0,9965</b> | 0,9885  | 0,9928        |
| Codec/18            | 0,9373 | 0,9749        | 0,9782  | <b>0,9950</b> |
| JacksonCore/26      | 0,8643 | <b>0,9623</b> | 0,8854  | 0,9480        |
| Lang/65             | 0,9342 | 0,9842        | 0,9674  | <b>0,9997</b> |

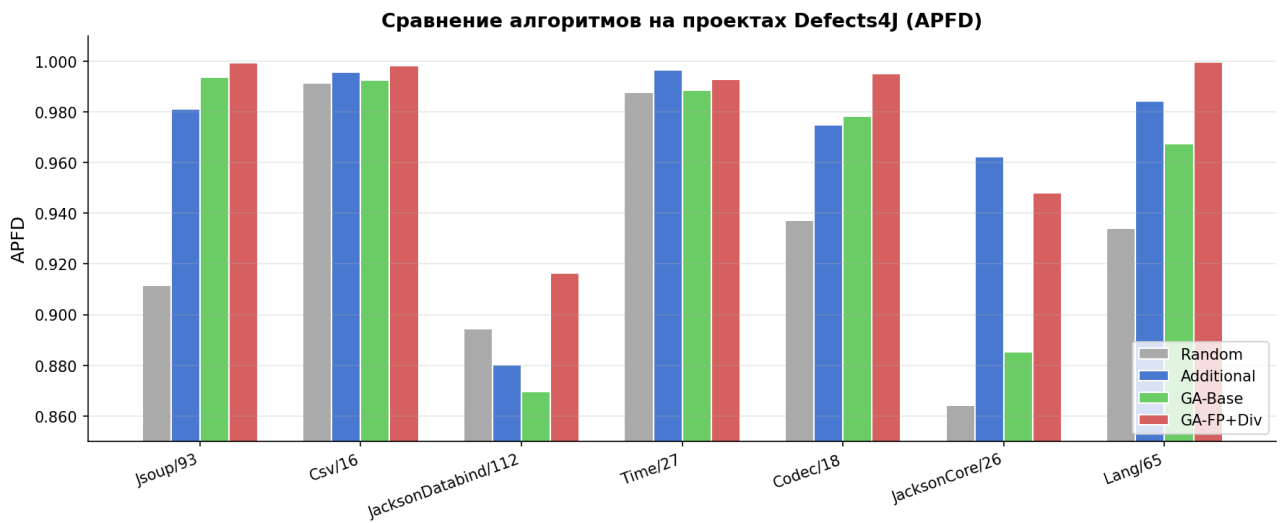


Рис.6.2. Сравнение алгоритмов на проектах Defects4J по метрике APFD

## 6.5 Анализ результатов

На проектах Defects4J алгоритм GA-FP+Div показывает наилучшие результаты на 5 из 7 проектов. На проектах Time/27 и JacksonCore/26 лучшим оказывается Additional. Это объясняется малым числом методов с ненулевой вероятностью дефектов по данным CIA-анализа: для Time/27 и JacksonCore/26 CIA предоставляет слабый сигнал, что снижает вклад компоненты вероятности дефектов в фитнес-функцию.

На проектах с богатыми CIA-данными (Codec/18, Lang/65, Jsoup/93) преимущество GA-FP+Div наиболее выражено. Например, на Codec/18 GA-FP+Div достигает APFD = 0,9994, превосходя Additional (0,9749) на 2,5 процентных пункта.

Алгоритм GA-Base уступает GA-FP+Div на всех проектах Defects4J, что подтверждает эффективность добавления компонент вероятности дефектов и структурного разнообразия в фитнес-функцию.

Таким образом, GA-FP+Div демонстрирует устойчивое преимущество над базовым генетическим алгоритмом, а его эффективность относительно жадного алгоритма определяется качеством CIA-данных для конкретного проекта.

## ЗАКЛЮЧЕНИЕ

В настоящей работе разработан и исследован алгоритм приоритизации тестовых случаев GA-FP+Div, представляющий собой генетический алгоритм с улучшенной многокомпонентной фитнес-функцией. Фитнес-функция учитывает три нормализованных критерия: покрытие кода, вероятность дефектов на основе CIA-анализа и структурное разнообразие тестов по расстоянию Жаккара.

В ходе работы получены следующие результаты:

- А. Проведён анализ существующих методов приоритизации тестовых случаев, выявлены ограничения жадных и эволюционных подходов, обоснована необходимость многокритериальной фитнес-функции.
- В. Разработана фитнес-функция, объединяющая покрытие кода, вероятность дефектов и структурное разнообразие тестов. Реализованы операторы ОХ-кроссовера, адаптивной мутации и элитизма.
- С. Выполнена программная реализация на языке Java с поддержкой загрузки данных в форматах BigFaultMatrix и Defects4J, а также сохранением результатов в форматах CSV и JSON.
- Д. Проведён сравнительный эксперимент на наборе данных BigFaultMatrix (4 вариации) и 7 проектах Defects4J. Алгоритм GA-FP+Div показал наилучшие результаты на 5 из 7 проектов Defects4J, превзойдя базовый генетический алгоритм на всех проектах.
- Е. Установлено что эффективность GA-FP+Div определяется качеством CIA-данных: на проектах с богатым CIA-анализом (Codec/18, Lang/65) преимущество алгоритма наиболее выражено.

Направления дальнейших исследований включают: автоматический подбор весов фитнес-функции методами настройки гиперпараметров; расширение бенчмарка за счёт дополнительных проектов Defects4J; исследование влияния параметра  $k$  на качество приоритизации.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Paygude P., Joshi S. D., Joshi M. Fault Aware Test Case Prioritization in Regression Testing using Genetic Algorithm // International Journal of Emerging Trends in Engineering Research. — 2020. — Vol. 8, No. 5. — P. 2112–2117. — DOI: 10.30534/ijeter/2020/104852020.
2. Mahdiah M., Mirian-Hosseiniabadi S.-H., Etemadi K., Bohlouli M. Incorporating Fault-Proneness Estimations into Coverage-Based Test Case Prioritization // Information and Software Technology. — 2021. — Vol. 133. — P. 106483. — DOI: 10.1016/j.infsof.2021.106483.
3. Elbaum S., Malishevsky A. G., Rothermel G. Test Case Prioritization: A Family of Empirical Studies // IEEE Transactions on Software Engineering. — 2002. — Vol. 28, No. 2. — P. 159–182. — DOI: 10.1109/32.988497.
4. Руденко М. А. tcp-methods: репозиторий данных для приоритизации тестовых случаев // GitHub. — URL: <https://github.com/oneren38/tcp-methods> (дата обращения: 08.04.2026).
5. dathpo. Test Case Prioritisation — Genetic Algorithm: BigFaultMatrix // GitHub. — URL: [https://github.com/dathpo/Test\\_Case\\_Prioritisation\\_-\\_Genetic\\_Al](https://github.com/dathpo/Test_Case_Prioritisation_-_Genetic_Al) (дата обращения: 08.04.2026).

## Программный код

```
package com.tcp.algorithm;

import com.tcp.model.TestCase;
5 import java.util.*;

/**
 * базовый жадный алгоритм
10 * на каждом шаге выбирает тест, покрывающий наибольшее количе
    ство
 */

public class AdditionalPrioritizer implements Prioritizer {

15     @Override
    public List<TestCase> prioritize(List<TestCase> testPool)
    {
        List<TestCase> result = new ArrayList<>();
        Set<String> coveredMethods = new HashSet<>();
        //копия пула, чтобы не модифицировать исходный список
20 List<TestCase> remaining = new ArrayList<>(testPool);

        while (!remaining.isEmpty()) {
            TestCase bestTest = null;
            int maxNewCoverage = -1;

25            //поиск теста с максимальным новым покрытием
            for (TestCase test : remaining) {
                int newCount = 0;
                for (String method : test.getCoveredMethods())
                {
30                    if (!coveredMethods.contains(method)) {
                        newCount++;
                    }
                }

                if (newCount > maxNewCoverage) {
35                    maxNewCoverage = newCount;
                    bestTest = test;
                }
            }
        }
    }
}
```

```

        } else if (newCount == maxNewCoverage &&
            bestTest != null) {
            //если покрытие одинаковое берем тест с бо
            льшей fault proneness
            if (test.getFaultProneness() > bestTest.
                getFaultProneness()) {
                bestTest = test;
            }
        }
    }
    //fallback: если все оставшиеся тесты не покрывают
    ничего нового
    if (bestTest == null) {
        bestTest = remaining.get(0);
    }

    result.add(bestTest);
    remaining.remove(bestTest);
    coveredMethods.addAll(bestTest.getCoveredMethods()
        );
    }
    return result;
}

@Override
public String getName() {
    return "Additional";
}
}

package com.tcp.algorithm;

import com.tcp.fitness.GaFitnessFunction;
import com.tcp.metric.ApfdCalculator;
import com.tcp.model.TestCase;

import java.util.*;

/**
 * Генетический алгоритм приоритизации тестов.
 *
 * Поддерживает два режима фитнес-функции:

```

```

* 1. Прямой APFD -- если переданы testToFaults и
* totalFaults (как в статье Paugude et al.)
15 * 2. GaFitnessFunction -- взвешенная сумма coverage +
* faultProneness + diversity
*
* Операторы:
* инициализация: случайное перемешивание
* селекция: турнирная (tournament selection)
20 * кроссовер: ОХ (Order Crossover)
* мутация: адаптивное число swap = sqrt(n)
* элитизм: лучшая особь переходит без изменений
*/
public class GeneticPrioritizer implements Prioritizer {
25
    private static final int TOURNAMENT_SIZE = 3;

    private final int populationSize;
    private final int generations;
30 private final double crossoverRate;
    private final double mutationRate;
    private final GaFitnessFunction fitnessFunction; // null е
        сли используется APFD-режим
    private final Map<String, Set<String>> testToFaults; //
        null если используется GaFitnessFunction
    private final int totalFaults;
35 private final Random rng = new Random(42);

    // Конструктор для режима GaFitnessFunction (coverage +
        faultProneness + diversity)
    public GeneticPrioritizer(int populationSize, int
        generations,
                                double crossoverRate, double
                                mutationRate,
40                                GaFitnessFunction
                                    fitnessFunction) {
        this(populationSize, generations, crossoverRate,
            mutationRate, fitnessFunction, null, 0);
    }

    // Конструктор для режима прямого APFD (как в статье
        Paugude et al.)
45 public GeneticPrioritizer(int populationSize, int
    generations,
                                double crossoverRate, double
                                mutationRate,

```

```

GaFitnessFunction
    fitnessFunction,
    Map<String, Set<String>>
        testToFaults,
        int totalFaults) {
50     this.populationSize = populationSize;
        this.generations = generations;
        this.crossoverRate = crossoverRate;
        this.mutationRate = mutationRate;
        this.fitnessFunction = fitnessFunction;
55     this.testToFaults = testToFaults;
        this.totalFaults = totalFaults;
    }

    // Выбирает режим фитнеса: прямой APFD или
    GaFitnessFunction
60 private double fitness(List<TestCase> seq) {
    if (testToFaults != null && totalFaults > 0)
        return ApfdCalculator.calculate(seq, testToFaults,
            totalFaults);
    return fitnessFunction.calculateFitness(seq);
    }

65 @Override
    public List<TestCase> prioritize(List<TestCase> testPool)
    {
        if (testPool.isEmpty()) return new ArrayList<>();

70     // адаптивная интенсивность мутации: sqrt(n) swap
        final int mutationIntensity = Math.max(1, (int) Math.
            sqrt(testPool.size()));

        List<List<TestCase>> population = initPopulation(
            testPool);
        double[] fit = evalAll(population);

75     int bestIdx = argmax(fit);
        List<TestCase> bestEver = new ArrayList<>(population.
            get(bestIdx));
        double bestFitness = fit[bestIdx];

80     for (int gen = 0; gen < generations; gen++) {
        List<List<TestCase>> next = new ArrayList<>(
            populationSize);
        double[] nextFit = new double[populationSize];
    }

```

```

85      // элимузм
      next.add(new ArrayList<>(bestEver));
      nextFit[0] = bestFitness;

      while (next.size() < populationSize) {
          List<TestCase> child1 = new ArrayList<>(
              tournament(population, fit));
          List<TestCase> child2 = new ArrayList<>(
              tournament(population, fit));

          boolean crossed = false;
          if (rng.nextDouble() < crossoverRate) {
              oxCrossover(child1, child2);
              crossed = true;
          }

          boolean mutated1 = false, mutated2 = false;
          if (rng.nextDouble() < mutationRate) { mutate(
              child1, mutationIntensity); mutated1 = true
              ; }
          if (rng.nextDouble() < mutationRate) { mutate(
              child2, mutationIntensity); mutated2 = true
              ; }

          int i = next.size();
          next.add(child1);
          nextFit[i] = (crossed || mutated1)
105              ? fitness(child1)
              : fit[indexOf(population, child1)];

          if (next.size() < populationSize) {
              int j = next.size();
              next.add(child2);
              nextFit[j] = (crossed || mutated2)
110                  ? fitness(child2)
                  : fit[indexOf(population, child2)
                      ];
          }
115     }

    population = next;
    fit = nextFit;

120     int idx = argmax(fit);

```

```

        if (fit[idx] > bestFitness) {
            bestFitness = fit[idx];
            bestEver = new ArrayList<>(population.get(idx)
                );
        }
125     }

    return bestEver;
}

130 private List<List<TestCase>> initPopulation(List<TestCase>
    pool) {
    List<List<TestCase>> pop = new ArrayList<>(
        populationSize);
    for (int i = 0; i < populationSize; i++) {
        List<TestCase> ind = new ArrayList<>(pool);
        Collections.shuffle(ind, rng);
135     pop.add(ind);
    }
    return pop;
}

140 private double[] evalAll(List<List<TestCase>> population)
    {
        double[] f = new double[population.size()];
        for (int i = 0; i < population.size(); i++)
            f[i] = fitness(population.get(i));
        return f;
145     }

    private List<TestCase> tournament(List<List<TestCase>>
        population, double[] fit) {
        int best = rng.nextInt(population.size());
        for (int i = 1; i < TOURNAMENT_SIZE; i++) {
150             int candidate = rng.nextInt(population.size());
            if (fit[candidate] > fit[best]) best = candidate;
        }
        return population.get(best);
    }

155 /**
 * OX (Order Crossover): сохраняет валидную перестановку.
 * Сегмент [start, end] из p1 копируется в child1,
 * остаток заполняется из p2 в порядке появления. Аналогич
 * но для child2.

```

```

160      */
private void oxCrossover(List<TestCase> p1, List<TestCase>
    p2) {
    int size = p1.size();
    if (size < 2) return;

165    int start = rng.nextInt(size);
    int end = rng.nextInt(size);
    if (start > end) { int tmp = start; start = end; end =
        tmp; }
    if (start == end) end = Math.min(end + 1, size - 1);

170    List<TestCase> c1 = buildOxChild(p1, p2, start, end);
    List<TestCase> c2 = buildOxChild(p2, p1, start, end);
    p1.clear(); p1.addAll(c1);
    p2.clear(); p2.addAll(c2);
}

175 private List<TestCase> buildOxChild(List<TestCase> donor,
    List<TestCase> filler, int start, int end) {
    int size = donor.size();
    List<TestCase> child = new ArrayList<>(Collections.
        nCopies(size, null));
    Set<String> inSegment = new HashSet<>();

180    for (int i = start; i <= end; i++) {
        child.set(i, donor.get(i));
        inSegment.add(donor.get(i).getId());
    }

185    Queue<TestCase> rest = new ArrayDeque<>();
    for (TestCase t : filler)
        if (!inSegment.contains(t.getId())) rest.add(t);

190    for (int i = 0; i < size; i++)
        if (child.get(i) == null) child.set(i, rest.poll()
            );

    return child;
}

195 private void mutate(List<TestCase> ind, int intensity) {
    for (int k = 0; k < intensity; k++)
        Collections.swap(ind, rng.nextInt(ind.size()), rng
            .nextInt(ind.size()));
}

```

```

    }

200 private int argmax(double[] arr) {
        int best = 0;
        for (int i = 1; i < arr.length; i++)
            if (arr[i] > arr[best]) best = i;
205         return best;
    }

    private int indexOf(List<List<TestCase>> population, List<
        TestCase> ind) {
        for (int i = 0; i < population.size(); i++)
210             if (population.get(i) == ind) return i;
        return 0;
    }

    @Override
215     public String getName() { return "GeneticPrioritizer"; }
}

package com.tcp.algorithm;

import com.tcp.model.TestCase;
5 import java.util.List;

/*
интерфейс для всех алгоритмов приоритизации
позволяет единообразно вызывать любые стратегии
10 */

public interface Prioritizer {
    /**
        * основной метод
        * принимает неупорядоченный пул тестов и возвращает списо
        к, отсортированный по приоритету
15 *
        * @param testPool - исходный список тестов
        * @return упорядоченный список тестов
        */
    List<TestCase> prioritize(List<TestCase> testPool);

    /**
        * возвращает название алгоритма для логирования и отчетов
        * @return имя метода приоритизации
        */
25     String getName();
}

```

```

}

package com.tcp.algorithm;

import com.tcp.model.TestCase;
5
import java.util.*;

/**
 * Random Search -- случайная перестановка тестов.
10 * Используется как baseline для сравнения с оптимизационными
   * алгоритмами.
   *
   * Для воспроизводимости результатов используется фиксированный seed.
   * APFD усредняется по нескольким прогонам в Main.
   */
15 public class RandomPrioritizer implements Prioritizer {

    private final Random rng;

    // seed для воспроизводимости
20 public RandomPrioritizer(long seed) {
        this.rng = new Random(seed);
    }

    @Override
25 public List<TestCase> prioritize(List<TestCase> testPool)
    {
        List<TestCase> result = new ArrayList<>(testPool);
        Collections.shuffle(result, rng);
        return result;
    }

30
    @Override
    public String getName() { return "Random"; }
}

package com.tcp.data;

import com.tcp.model.TestCase;
5
import java.io.*;
import java.util.*;
import java.util.stream.Collectors;

```

```

10 public class DataLoader {

    public static class FaultMatrixData {
        public final List<TestCase> tests;
        public final Map<String, Set<String>> testToFaults;
15         public final int totalFaults;

        public FaultMatrixData(List<TestCase> tests, Map<
            String, Set<String>> testToFaults, int totalFaults)
        {
            this.tests = tests;
            this.testToFaults = testToFaults;
20             this.totalFaults = totalFaults;
        }
    }

    // --- BigFaultMatrix: testId,0,1,0,... ---

25     public static FaultMatrixData loadFaultMatrix(String
        filePath) throws Exception {
        List<TestCase> tests = new ArrayList<>();
        Map<String, Set<String>> testToFaults = new HashMap
            <>();
        int totalFaults = -1;

30         for (String line : readLines(filePath)) {
            String[] parts = line.split(",");
            if (parts.length < 2) continue;
            if (totalFaults == -1) totalFaults = parts.length
                - 1;

35             String testId = parts[0].trim();
            Set<String> killed = new HashSet<>();
            for (int i = 1; i < parts.length; i++)
                if ("1".equals(parts[i].trim())) killed.add("F
                    " + (i - 1));

40             double fp = totalFaults > 0 ? (double) killed.size
                () / totalFaults : 0.0;
            tests.add(new TestCase(testId, killed, fp, 100.0))
                ;
            testToFaults.put(testId, killed);

45         }
    }
}

```

```

        return new FaultMatrixData(tests, testToFaults, Math.
            max(totalFaults, 0));
    }

    public static FaultMatrixData createVariation(
        FaultMatrixData original, double flipRatio, long seed)
    {
50         Random rng = new Random(seed);
        List<TestCase> newTests = new ArrayList<>();
        Map<String, Set<String>> newMap = new HashMap<>();

        for (TestCase t : original.tests) {
55             Set<String> faults = new HashSet<>(original.
                testToFaults.get(t.getId()));
            for (int f = 0; f < original.totalFaults; f++) {
                if (rng.nextDouble() < flipRatio) {
                    String id = "F" + f;
                    if (!faults.add(id)) faults.remove(id);
60                 }
            }
            double fp = original.totalFaults > 0 ? (double)
                faults.size() / original.totalFaults : 0.0;
            newTests.add(new TestCase(t.getId(), faults, fp,
                100.0));
            newMap.put(t.getId(), faults);
65         }

        return new FaultMatrixData(newTests, newMap, original.
            totalFaults);
    }

70     // --- Defects4J / tcp-methods ---
    // Struktura papki projectPath:
    //   coverage/matrix.csv           -- zagolovok: test,
       method1,...; stroki: testId,0,1,...
    //   mutants/kill.csv             -- MutantNo,[FAIL|TIME|
       EXC|LIVE|UNCOV]
    //   mutants/testMap.csv          -- TestNo,TestName,
       Runtime
75    //   mutants/covMap.csv           -- TestNo,MutantNo
    //   cia/impact_probabilities.csv -- owner,method,
       descriptor,roc,fwd,impact

    public static FaultMatrixData loadDefects4JData(String
        projectPath) throws Exception {

```

```

List<TestCase> tests = loadCoverage(projectPath + "/"
    coverage/matrix.csv");
80 Map<String, Set<String>> testToFaults = loadKills(
    projectPath + "/mutants/kill.csv",
    projectPath + "/mutants/testMap.csv",
    projectPath + "/mutants/covMap.csv",
    tests);
85 tests = applyImpact(tests, projectPath + "/cia/
    impact_probabilities.csv");

int totalFaults = (int) testToFaults.values().stream()
    .flatMap(Set::stream).distinct().count();
System.out.printf("  Tests: %d, mutants: %d%n", tests.
    size(), totalFaults);
return new FaultMatrixData(tests, testToFaults,
    totalFaults);
90 }

private static List<TestCase> loadCoverage(String path)
throws Exception {
    List<TestCase> tests = new ArrayList<>();
    List<String> methods = new ArrayList<>();
95

    try (BufferedReader br = new BufferedReader(new
        FileReader(new File(path)))) {
        String line;
        boolean header = true;
        while ((line = br.readLine()) != null) {
100 line = line.trim();
            if (line.isEmpty()) continue;
            String[] parts = splitCsv(line);
            if (header) {
                // первая строка - заголовок с именами мет
                одов
105 for (int i = 1; i < parts.length; i++)
                    methods.add(parts[i].replace("\"", ""))
                        .trim());
                header = false;
                continue;
            }
110 Set<String> covered = new HashSet<>();
            for (int i = 1; i < parts.length && i - 1 <
                methods.size(); i++)
                if ("1".equals(parts[i].trim())) covered.
                    add(methods.get(i - 1));

```

```

        tests.add(new TestCase(parts[0].trim(),
                                covered, 0.0, 100.0));
    }
115     }
    return tests;
}

private static Map<String, Set<String>> loadKills(String
120     killPath, String testMapPath,
                                String
                                covMapPath
                                ,
                                List<
                                TestCase
                                >
                                tests
                                )
                                throws
                                Exception
                                {
        Map<String, Set<String>> result = new HashMap<>();
        for (TestCase t : tests) result.put(t.getId(), new
            HashSet<>());

        // kill.csv: MutantNo -> status (FAIL/TIME/EXC =
        killed)
125     Set<String> killed = new HashSet<>();
        for (String line : readLines(killPath)) {
            String[] p = line.split(",");
            if (p.length < 2) continue;
            String status = p[1].trim();
130             if (status.equals("FAIL") || status.equals("TIME")
                || status.equals("EXC"))
                killed.add(p[0].trim());
        }

        // testMap.csv: TestNo -> TestName (className only, no
        method)
135     Map<String, String> testNoToName = new HashMap<>();
        for (String line : readLines(testMapPath)) {
            String[] p = line.split(",");
            if (p.length < 2) continue;
            testNoToName.put(p[0].trim(), p[1].trim());
140     }

```

```

// coverage keys look like "org.jsoup.FormElementTest#
//      methodName"
// testMap has only "org.jsoup.FormElementTest" (class
//      name, no method)
// build index: className -> list of full testIds from
//      coverage
145 Map<String, List<String>> classToIds = new HashMap<>()
    ;
    for (String testId : result.keySet()) {
        String cls = testId.contains("#") ? testId.
            substring(0, testId.indexOf('#')) : testId;
        classToIds.computeIfAbsent(cls, k -> new ArrayList
            <>()).add(testId);
    }
150
// covMap.csv: TestNo, MutantNo - assign killed mutants
//      to tests
    for (String line : readLines(covMapPath)) {
        String[] p = line.split(",");
        if (p.length < 2) continue;
155 String testNo = p[0].trim(), mutantNo = p[1].trim
        ();
        if (!killed.contains(mutantNo)) continue;
        String className = testNoToName.get(testNo);
        if (className == null) continue;
        for (String testId : classToIds.getOrDefault(
            className, Collections.emptyList()))
160         result.get(testId).add("M" + mutantNo);
    }

    return result;
}
165

private static List<TestCase> applyImpact(List<TestCase>
    tests, String path) throws Exception {
    if (!new File(path).exists()) return tests;

    // Hopman узывает ключи CIA: org/jsoup/nodes/FormElement
    //      , <init>, ... -> org.jsoup.nodes.FormElement#init
170 Map<String, Double> scores = new HashMap<>();
    for (String line : readLines(path)) {
        String[] p = splitCsv(line);
        if (p.length < 6) continue;
        try {

```

```

175         String className = p[0].trim().replace('/', '.');
        // убираем угловые скобки: <init> -> init, <
        // clinit> -> clinit
        String methodName = p[1].trim().replace("<", "
        ").replace(">", "");
        String key = className + "#" + methodName;
        double score = Double.parseDouble(p[5].trim())
        ;
180         scores.merge(key, score, Math::max);
    } catch (NumberFormatException ignored) {}
}

// Нормализуем ключи coverage до того же формата:
185 // "org.jsoup.parser$Parser#Parser(org.jsoup.parser.
    // TreeBuilder):24" -> "org.jsoup.parser.Parser#Parser
    "

return tests.stream().map(t -> {
    double sum = 0.0; int count = 0;
    for (String m : t.getCoveredMethods()) {
        String key = normalizeCoverageKey(m);
190         if (scores.containsKey(key)) { sum += scores.
            get(key); count++; }
    }
    return new TestCase(t.getId(), t.getCoveredMethods
        (),
        count > 0 ? sum / count : 0.0, t.
            getExecutionTimeMs());
}).collect(Collectors.toList());
195 }

// "org.jsoup.parser$Parser#Parser(org.jsoup.parser.
    // TreeBuilder):24" -> "org.jsoup.parser.Parser#Parser"
private static String normalizeCoverageKey(String raw) {
    String s = raw.replaceAll(":\d+$", ""); // убираем
    :24
200     int paren = s.indexOf('(');
    if (paren >= 0) s = s.substring(0, paren); // убираем
        параметры
    return s.replace('$', '.'); // $ -> .
        для вложенных классов
}

205 // --- Synthetic data ---

```

```

210  /**
    * Генерирует синтетические данные с независимым риском и
    * покрытием.
    *
    * Тесты делятся на три группы чтобы GA-FP+Div имел реальн
    * ое преимущество:
    *   (30%): большое покрытие, мало фолтов -- жадный выбира
    * ет их первыми.
    *   (20%): малое покрытие, высокий риск и много ключевых
    * фолтов.
    *   Обычные (50%): покрытие и фолты пропорциональны
    * density.
215  */
    public static FaultMatrixData generateSynthetic(int
        numTests, int numFaults, double density) {
        List<TestCase> tests = new ArrayList<>();
        Map<String, Set<String>> map = new HashMap<>();
        Random rng = new Random(42);
220
        // первые 40% фолтов считаются "рискованными"
        int riskFaults = (int)(numFaults * 0.4);

        for (int t = 0; t < numTests; t++) {
225         Set<String> faults = new HashSet<>();
            double fp;
            int execTime = 50 + rng.nextInt(150);
            double roll = rng.nextDouble();

230         if (roll < 0.30) {
            // "Ловушка": много обычных фолтов, но не риск
            // ованных -- низкий риск
            for (int f = riskFaults; f < numFaults; f++)
                if (rng.nextDouble() < density * 2.5)
                    faults.add("F" + f);
            fp = 0.05 + rng.nextDouble() * 0.05;
235
        } else if (roll < 0.50) {
            // "Жемчужина": мало покрытия, но попадает в р
            // искованные фолты
            for (int f = 0; f < riskFaults; f++)
                if (rng.nextDouble() < density * 1.5)
                    faults.add("F" + f);
240            fp = 0.6 + rng.nextDouble() * 0.4;
        }
    }

```

```

    } else {
        // Обычный тест
        for (int f = 0; f < numFaults; f++)
            if (rng.nextDouble() < density) faults.add
245         ("F" + f);
        fp = numFaults > 0 ? (double) faults.size() /
            numFaults : 0.0;
    }

    TestCase tc = new TestCase("T" + t, faults, fp,
        execTime);
250    tests.add(tc);
    map.put(tc.getId(), new HashSet<>(faults));
}

    return new FaultMatrixData(tests, map, numFaults);
255 }

// --- Utils ---

// Read file line by line, skip empty lines and header (
// first line)
260 private static List<String> readLines(String path) throws
    Exception {
        List<String> lines = new ArrayList<>();
        File f = new File(path);
        if (!f.exists()) return lines;
        try (BufferedReader br = new BufferedReader(new
            FileReader(f))) {
265             String line;
            boolean header = true;
            while ((line = br.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty()) continue;
270                 if (header) { header = false; continue; }
                lines.add(line);
            }
        }
        return lines;
275 }

// Split CSV line respecting commas inside quotes
private static String[] splitCsv(String line) {
    List<String> tokens = new ArrayList<>();
280     StringBuilder sb = new StringBuilder();

```

```

        boolean inQuotes = false;
        for (char c : line.toCharArray()) {
            if (c == '"'') inQuotes = !inQuotes;
            else if (c == ',' && !inQuotes) { tokens.add(sb.
285         toString()); sb.setLength(0); }
            else sb.append(c);
        }
        tokens.add(sb.toString());
        return tokens.toArray(new String[0]);
    }
290 }

package com.tcp.fitness;

import com.tcp.model.TestCase;
5
import java.util.HashSet;
import java.util.List;
import java.util.Set;

10 /**
 * Фитнесс-функция для генетического алгоритма.
 *
 * Оценивает качество порядка тестов по трём компонентам, кажд
    ая в [0, 1]:
 *   coverage -- доля покрытых уникальных методов от всех в пу
    ле
15 *   faultProneness -- средний риск первых TOP_K тестов
 *   diversity -- среднее расстояние Жаккара между соседними п
    арами в топ-TOP_K
 *
 * Итоговый фитнесс = взвешенная сумма трёх компонент (веса за
    даются в конструкторе).
 */
20 public class GaFitnessFunction {

    // число тестов в начале последовательности, учитываемых д
    ля risk и diversity
    private static final int TOP_K = 10;

25     private final double weightCoverage;
    private final double weightFaultProneness;
    private final double weightDiversity;

```

```

public GaFitnessFunction(double weightCoverage, double
    weightFaultProneness, double weightDiversity) {
30     this.weightCoverage = weightCoverage;
        this.weightFaultProneness = weightFaultProneness;
        this.weightDiversity = weightDiversity;
    }

35     // Вычисляет фитнес последовательности.
    // totalMethods вычисляется каждый раз из нуля - безопасно
    при смене проектов.
    public double calculateFitness(List<TestCase> seq) {
        if (seq.isEmpty()) return 0.0;

40         Set<String> all = new HashSet<>();
        for (TestCase t : seq) all.addAll(t.getCoveredMethods
            ());
        int totalMethods = Math.max(all.size(), 1);

        return weightCoverage      * coverage(seq,
            totalMethods)
45             + weightFaultProneness * faultProneness(seq)
            + weightDiversity      * diversity(seq);
    }

    // Доля уникальных покрытых методов от totalMethods
50     private double coverage(List<TestCase> seq, int
        totalMethods) {
        Set<String> covered = new HashSet<>();
        for (TestCase t : seq) covered.addAll(t.
            getCoveredMethods());
        return (double) covered.size() / totalMethods;
    }

55     // Средняя fault-proneness первых TOP_K тестов
    private double faultProneness(List<TestCase> seq) {
        int k = Math.min(seq.size(), TOP_K);
        double sum = 0.0;
60         for (int i = 0; i < k; i++) sum += seq.get(i).
            getFaultProneness();
        return sum / k;
    }

    // Среднее расстояние Жаккара между соседними парами в топ
    -TOP_K
65     private double diversity(List<TestCase> seq) {

```

```

        int k = Math.min(seq.size(), TOP_K);
        if (k < 2) return 0.0;
        double sum = 0.0;
        for (int i = 0; i < k - 1; i++)
            sum += jaccard(seq.get(i).getCoveredMethods(), seq
70         .get(i + 1).getCoveredMethods());
        return sum / (k - 1);
    }

    // Расстояние Жаккара: 0 = одинаковые, 1 = полностью разные
    множества
75 private double jaccard(Set<String> a, Set<String> b) {
        if (a.isEmpty() && b.isEmpty()) return 0.0;
        if (a.isEmpty() || b.isEmpty()) return 1.0;

        // считаем пересечение без создания нового множества
80         int intersection = 0;
        for (String s : a) if (b.contains(s)) intersection++;
        int union = a.size() + b.size() - intersection;

        return 1.0 - (double) intersection / union;
85     }
}

package com.tcp.metric;

import com.tcp.model.TestCase;
5 import java.util.*;

/**
 * Метрика APFD (Average Percentage of Faults Detected).
10 * Оценивает скорость обнаружения дефектов: чем ближе к 1, тем
    раньше находятся мутанты.
    *
    * Формула:  $APFD = 1 - \frac{sumTF}{(n * m) + 1} / (2n)$ 
    * n -- число тестов
    * m -- число мутантов
15 * sumTF -- сумма позиций первых тестов, обнаруживших каждый
    мутант
    */
public class ApfdCalculator {

    public static double calculate(List<TestCase> orderedTests
    ,

```

```

20         Map<String, Set<String>>
           testToMutants,
           int totalMutants) {
    int n = orderedTests.size();
    int m = totalMutants;
    if (n == 0 || m == 0) return 0.0;
25
    Set<String> detected = new HashSet<>();
    double sumTF = 0.0;

    for (int i = 0; i < n; i++) {
30        for (String mutant : testToMutants.getDefault(
            orderedTests.get(i).getId(), Collections.
            emptySet())) {
            if (detected.add(mutant)) sumTF += (i + 1);
        }
    }

35    return 1.0 - sumTF / ((double) n * m) + 1.0 / (2.0 * n
        );
    }
}

```

```

package com.tcp.metric;

import com.tcp.model.TestCase;
5
import java.util.*;

/**
 * Метрика APFDc (APFD with Cost) - APFD с учётом времени выпол
   * нения тестов.
10
 *
 * В отличие от APFD, взвешивает обнаружение мутантов по кумул
   * ятивному времени:
 * мутант, найденный быстрым тестом в начале, ценится выше чем
   * найденный медленным в конце.
 *
 * Формула:  $APFDc = \frac{\sum (totalTime - cumTime_i + execTime_i)}{2} / (totalTime * m)$ 
15
 * totalTime -- суммарное время всех тестов
 * cumTime_i -- кумулятивное время до конца теста i
 * m -- число мутантов
 */
public class ApfdCostCalculator {

```

```

20 public static double calculateWithCost(List<TestCase>
    orderedTests,
                                           Map<String, Set<
                                           String>>
                                           testToMutants,
                                           int totalMutants) {
    if (orderedTests.isEmpty() || totalMutants == 0)
        return 0.0;

25     double totalTime = orderedTests.stream().mapToDouble(
        TestCase::getExecutionTimeMs).sum();
    Set<String> detected = new HashSet<>();
    double weightedSum = 0.0;
    double cumTime = 0.0;

30     for (TestCase test : orderedTests) {
        cumTime += test.getExecutionTimeMs();
        for (String mutant : testToMutants.getOrDefault(
            test.getId(), Collections.emptySet())) {
            if (detected.add(mutant))
35                 weightedSum += totalTime - cumTime + test.
                    getExecutionTimeMs() / 2.0;
        }
    }

    return weightedSum / (totalTime * totalMutants);
40 }
}

    package com.tcp;

    import com.tcp.algorithm.AdditionalPrioritizer;
5    import com.tcp.algorithm.GeneticPrioritizer;
    import com.tcp.algorithm.Prioritizer;
    import com.tcp.algorithm.RandomPrioritizer;
    import com.tcp.data.DataLoader;
    import com.tcp.data.DataLoader.FaultMatrixData;
10    import com.tcp.fitness.GaFitnessFunction;
    import com.tcp.metric.ApfdCalculator;
    import com.tcp.metric.ApfdCostCalculator;
    import com.tcp.model.TestCase;

15    import java.io.*;
    import java.time.LocalDateTime;

```

```

import java.time.format.DateTimeFormatter;
import java.util.*;

20 /**
   * Точка входа бенчмарка приоритизации регрессионных тестов.
   *
   * Алгоритмы:
   *   Random -- случайная перестановка (среднее по 5 прогонам)
25 *   Additional -- жадный baseline по покрытию
   *   GA-Base -- ГА только с покрытием (coverage weight = 1.0)
   *   GA-FP+Div -- ГА с fault-proneness и диверсификацией (0.2
       / 0.6 / 0.2)
   *
   * Метрики: APFD, APFDc
30 * Результаты сохраняются в results/results.csv и results/
   results.json
   */
public class Main {

    private static final String BFM_PATH      = "data/
        bigfaultmatrix/bigfaultmatrix.txt";
35 private static final String RESULTS_DIR    = "results";
    private static final String CSV_PATH      = RESULTS_DIR + "/"
        results.csv";
    private static final String JSON_PATH     = RESULTS_DIR + "/"
        results.json";
    private static final int      RANDOM_RUNS = 5;

40 private static final List<String> DEFAULT_PROJECTS = List.
    of(
        "Jsoup/93", "Csv/16", "JacksonDatabind/112",
        "Time/27", "Codec/18", "JacksonCore/26", "Lang/65"
    );

45 // Хранит все результаты текущей сессии для записи в файлы
    private static final List<ResultRow> allResults = new
        ArrayList<>();

    public static void main(String[] args) {
        initResultsDir();
50 Scanner sc = new Scanner(System.in);
        System.out.println("=== TCP Benchmark ===");

        while (true) {

```

```

55      System.out.println("\n1. BigFaultMatrix (4 вариаци
        и)");
      System.out.println("2. Defects4J");
      System.out.println("3. Синтетические данные");
      System.out.println("4. Все Defects4J проекты");
      System.out.println("0. Выход");
      System.out.print("Выбор: ");
60      String choice = sc.nextLine().trim();

      switch (choice) {
          case "1": runBigFaultMatrix(); break;
          case "2":
65              System.out.print("Путь к проекту (например
                  data/raw/Jsoup/93): ");
              String path = sc.nextLine().trim();
              runBenchmark(path, loadOrNull(() ->
                  DataLoader.loadDefects4JData(path)));
              break;
          case "3": runBenchmark("synthetic",
              runSynthetic(sc)); break;
70          case "4": runAllProjects(sc); break;
          case "0":
              saveResults();
              sc.close();
              return;
75          default: System.out.println("Введите 0-4.");
      }

      System.out.print("\nНажмите Enter...");
      sc.nextLine();
80  }
}

// --- Прогоны ---

85 private static void runBigFaultMatrix() {
    FaultMatrixData original = loadOrNull(() -> DataLoader
        .loadFaultMatrix(BFM_PATH));
    if (original == null) return;

    System.out.println("\n=== BigFaultMatrix: 4 вариации
        ===");

90    String[] labels = { "BFM-original", "BFM-flip10", "BFM-
        flip20", "BFM-flip30" };

```

```

double[] flips = { 0.0, 0.1, 0.2, 0.3 };
long[] seeds = { 0L, 1L, 2L, 3L };

95 System.out.printf("%n%-22s | %-8s | %-8s | %-8s | %-8s
    | %s%n",
        "Вариация", "Random", "Additional", "GA-Base",
        "GA-FP+Div", "Лучший");
System.out.println("-".repeat(82));

100 for (int v = 0; v < 4; v++) {
    FaultMatrixData data = (v == 0)
        ? original
        : DataLoader.createVariation(original,
            flips[v], seeds[v]);

    int gens = adaptiveGens(data.tests.size());
105 Map<String, double[]> metrics = collectMetrics(
    data, gens);

    double[] rnd = metrics.get("Random");
    double[] add = metrics.get("Additional");
    double[] gab = metrics.get("GA-Base");
110 double[] gaf = metrics.get("GA-FP+Div");

    String best = bestName(new double[]{rnd[0], add
        [0], gab[0], gaf[0]},
        new String[]{"Random", "Additional", "GA-
            Base", "GA-FP+Div"});

115 System.out.printf("%-22s | %-8.4f | %-8.4f | %-8.4
    f | %-8.4f | %s%n",
        labels[v], rnd[0], add[0], gab[0], gaf[0],
        best);

    allResults.add(new ResultRow(labels[v], data.tests
        .size(), data.totalFaults,
        rnd[0], rnd[1], add[0], add[1], gab[0],
        gab[1], gaf[0], gaf[1]));
120 }
System.out.println("-".repeat(82));
}

125 private static FaultMatrixData runSynthetic(Scanner sc) {
    int tests = readInt(sc, "Тестов (100): ", 100);
    int faults = readInt(sc, "Фолтов (50): ", 50);

```

```

double dens = readDouble(sc, "Плотность (0.1): ", 0.1)
;
return DataLoader.generateSynthetic(tests, faults,
    dens);
}

130 private static void runBenchmark(String label,
    FaultMatrixData data) {
    if (data == null || data.tests.isEmpty() || data.
        totalFaults == 0) {
        System.err.println("Нет данных для бенчмарка.");
        return;
135     }

    int gens = adaptiveGens(data.tests.size());
    System.out.printf("    Поколений ГА: %d\n", gens);

140 Map<String, double[]> metrics = collectMetrics(data,
    gens);
    printResults(metrics);

    double[] rnd = metrics.get("Random");
    double[] add = metrics.get("Additional");
145 double[] gab = metrics.get("GA-Base");
    double[] gaf = metrics.get("GA-FP+Div");

    allResults.add(new ResultRow(label, data.tests.size(),
        data.totalFaults,
            rnd[0], rnd[1], add[0], add[1], gab[0], gab
                [1], gaf[0], gaf[1]));
150 }

private static void runAllProjects(Scanner sc) {
    System.out.print("Базовый путь к данным (например data
        /raw): ");
    String base = sc.nextLine().trim();
155

    System.out.println("\n=== ПАКЕТНЫЙ ЗАПУСК ===");
    System.out.printf("%-25s | %-8s | %-8s | %-8s | %-8s |
        %s\n",
        "Проект", "Random", "Additional", "GA-Base", "
            GA-FP+Div", "Лучший");
    System.out.println("-".repeat(82));
160

    for (String project : DEFAULT_PROJECTS) {

```

```

FaultMatrixData data = loadOrNull(() -> DataLoader
    .loadDefects4JData(base + "/" + project));

if (data == null || data.tests.isEmpty() || data.
    totalFaults == 0) {
165     System.out.printf("%-25s | ПРОВУЩЕН%n",
        project);
        continue;
    }

    int gens = adaptiveGens(data.tests.size());
170 Map<String, double[]> metrics = collectMetrics(
        data, gens);

    double[] rnd = metrics.get("Random");
    double[] add = metrics.get("Additional");
    double[] gab = metrics.get("GA-Base");
175 double[] gaf = metrics.get("GA-FP+Div");

    String best = bestName(new double[]{rnd[0], add
        [0], gab[0], gaf[0]},
        new String[]{"Random", "Additional", "GA-
            Base", "GA-FP+Div"});

180 System.out.printf("%-25s | %-8.4f | %-8.4f | %-8.4
        f | %-8.4f | %s%n",
            project, rnd[0], add[0], gab[0], gaf[0],
            best);

    allResults.add(new ResultRow(project, data.tests.
        size(), data.totalFaults,
            rnd[0], rnd[1], add[0], add[1], gab[0],
            gab[1], gaf[0], gaf[1]));
185 }
    System.out.println("-".repeat(82));
}

// --- Сбор метрик ---

190 // Запускаем все алгоритмы и возвращаем Map: algorithmName
    -> [apfd, apfdc]
private static Map<String, double[]> collectMetrics(
    FaultMatrixData data, int gens) {
    Map<String, double[]> result = new LinkedHashMap<>();

```

```

195 // Random: среднее по RANDOM_RUNS прогонам
double sumApfd = 0, sumApfdc = 0;
for (int r = 0; r < RANDOM_RUNS; r++) {
    List<TestCase> ordered = new RandomPrioritizer(r).
        prioritize(new ArrayList<>(data.tests));
    sumApfd += ApfdCalculator.calculate(ordered, data
200 .testToFaults, data.totalFaults);
    sumApfdc += ApfdCostCalculator.calculateWithCost(
        ordered, data.testToFaults, data.totalFaults);
}
result.put("Random", new double[]{ sumApfd /
    RANDOM_RUNS, sumApfdc / RANDOM_RUNS });

// Additional, GA-Base, GA-FP+Div
205 List<String> names = List.of("Additional", "GA-Base",
    "GA-FP+Div");
List<Prioritizer> algorithms = new ArrayList<>();
algorithms.add(new AdditionalPrioritizer());
algorithms.add(new GeneticPrioritizer(100, gens, 0.8,
    0.2, new GaFitnessFunction(1.0, 0.0, 0.0)));
algorithms.add(new GeneticPrioritizer(100, gens, 0.8,
    0.2, new GaFitnessFunction(0.4, 0.4, 0.2)));
210
for (int i = 0; i < algorithms.size(); i++) {
    List<TestCase> ordered = algorithms.get(i).
        prioritize(new ArrayList<>(data.tests));
    double apfd = ApfdCalculator.calculate(ordered,
        data.testToFaults, data.totalFaults);
    double apfdc = ApfdCostCalculator.
        calculateWithCost(ordered, data.testToFaults,
        data.totalFaults);
215 result.put(names.get(i), new double[]{ apfd, apfdc
        });
    // сохраняем упорядоченный список для вывода топ-3
    orderedResults.put(names.get(i), ordered);
}

// Random топ-3: берём первый прогон
220 orderedResults.put("Random",
    new RandomPrioritizer(0).prioritize(new
        ArrayList<>(data.tests)));

return result;
}
225

// хранит упорядоченные списки тестов для вывода топ-3

```

```

private static final Map<String, List<TestCase>>
    orderedResults = new LinkedHashMap<>();

private static void printResults(Map<String, double[]>
    metrics) {
230     System.out.println("\n--- РЕЗУЛЬТАТЫ ---");
    System.out.printf("%-20s | %-8s | %-8s\n", "Алгоритм",
        "APFD", "APFDc");
    System.out.println("-".repeat(42));

    String bestName = null; double bestApfd = -1;
235     for (Map.Entry<String, double[]> e : metrics.entrySet()
        ()) {
        System.out.printf("%-20s | %-8.4f | %-8.4f\n", e.
            getKey(), e.getValue()[0], e.getValue()[1]);
        if (e.getValue()[0] > bestApfd) { bestApfd = e.
            getValue()[0]; bestName = e.getKey(); }
    }
    System.out.println("-".repeat(42));
240     System.out.printf("Лучший: %s (APFD = %.4f)\n",
        bestName, bestApfd);

    // Топ-3 местов для каждого алгоритма
    System.out.println("\n--- ТОП-3 МЕСТОВ ---");
    for (Map.Entry<String, List<TestCase>> e :
        orderedResults.entrySet()) {
245         System.out.printf("%n%s:%n", e.getKey());
        System.out.printf("  %-12s | %-8s | %-10s\n", "Tec
            t", "Risk", "Coverage");
        System.out.println("  " + "-".repeat(36));
        List<TestCase> ordered = e.getValue();
        for (int i = 0; i < Math.min(3, ordered.size()); i
            ++){
250             TestCase t = ordered.get(i);
            System.out.printf("  %-12s | %-8.3f | %d\n",
                t.getId(), t.getFaultProneness(),
                t.getCoveredMethods().size());
        }
255     }
    orderedResults.clear();
}

// --- Сохранение результатов ---
260
private static void saveResults() {

```

```

        if (allResults.isEmpty()) return;
        try {
            new File(RESULTS_DIR).mkdirs();
            saveCsv();
            saveJson();
            System.out.println("\nРезультаты сохранены:");
            System.out.println("  CSV:  " + CSV_PATH);
            System.out.println("  JSON: " + JSON_PATH);
        } catch (Exception e) {
            System.err.println("Ошибка сохранения: " + e.
                               getMessage());
        }
    }

    private static void saveCsv() throws Exception {
        // Используем ; как разделитель - Excel в русской лока
        // ли ожидает именно его
        // sep= в первой строке подсказывает Excel какой разде
        // литель использован
        boolean exists = new File(CSV_PATH).exists();
        try (PrintWriter pw = new PrintWriter(
            new OutputStreamWriter(new FileOutputStream(
                CSV_PATH, true), "UTF-8"))) {
            if (!exists) {
                pw.println("sep="); // директива для Excel -
                // автоматически выберет разделитель
                pw.println("timestamp;dataset;tests;faults;" +
                    "random_apfd;random_apfdc;" +
                    "additional_apfd;additional_apfdc;" +
                    "gabase_apfd;gabase_apfdc;" +
                    "gafpdiv_apfd;gafpdiv_apfdc");
            }
            String ts = LocalDateTime.now().format(
                DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:
                ss"));
            for (ResultRow r : allResults)
                pw.printf("%s;%s;%d;%d;%.4f;%.4f;%.4f;%.4f;%.4
                    f;%.4f;%.4f;%.4f%n",
                        ts, r.dataset, r.tests, r.faults,
                        r.randomApfd, r.randomApfdc,
                        r.addApfd, r.addApfdc,
                        r.gaBaseApfd, r.gaBaseApfdc,
                        r.gaFpApfd, r.gaFpApfdc);
        }
    }
}

```

```

300 private static void saveJson() throws Exception {
        String ts = LocalDateTime.now().format(
            DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"))
        ;
        StringBuilder sb = new StringBuilder("[\n");
        for (int i = 0; i < allResults.size(); i++) {
305         ResultRow r = allResults.get(i);
            sb.append("    {\n");
            sb.append(String.format("        \"timestamp\": \"%s\n",\n",\n", ts));
            sb.append(String.format("        \"dataset\": \"%s\",\n",\n", r.dataset));
            sb.append(String.format("        \"tests\": %d,\n", r.tests));
            sb.append(String.format("        \"faults\": %d,\n", r.faults));
310         sb.append(String.format("        \"Random\":      {\n",
            "            \"apfd\": %.4f, \"apfdc\": %.4f},\n", r.randomApfd, r.randomApfdc));
            sb.append(String.format("        \"Additional\": {\n",
            "            \"apfd\": %.4f, \"apfdc\": %.4f},\n", r.addApfd, r.addApfdc));
            sb.append(String.format("        \"GA-Base\":      {\n",
            "            \"apfd\": %.4f, \"apfdc\": %.4f},\n", r.gaBaseApfd, r.gaBaseApfdc));
            sb.append(String.format("        \"GA-FP+Div\": {\n",
            "            \"apfd\": %.4f, \"apfdc\": %.4f}\n", r.gaFpApfd, r.gaFpApfdc));
            sb.append(i < allResults.size() - 1 ? "    },\n" : "    }\n");
315         }
            sb.append("]");
            try (PrintWriter pw = new PrintWriter(new FileWriter(
                JSON_PATH))) {
                pw.print(sb);
            }
320     }

    // --- Вспомогательные методы ---

    private static int adaptiveGens(int n) {
325         return Math.max(50, (int)(Math.log(n) / Math.log(2)) *
            10);
    }

```

```

private static String bestName(double[] apfds, String[]
names) {
    int best = 0;
330     for (int i = 1; i < apfds.length; i++) if (apfds[i] >
        apfds[best]) best = i;
    return names[best];
}

private static void initResultsDir() {
335     new File(RESULTS_DIR).mkdirs();
}

private static FaultMatrixData loadOrNull(ThrowingSupplier
<FaultMatrixData> supplier) {
    try { return supplier.get(); }
340     catch (Exception e) { System.err.println("Ошибка загрузки
        зки: " + e.getMessage()); return null; }
}

private static int readInt(Scanner sc, String prompt, int
defaultVal) {
    System.out.print(prompt);
345     String s = sc.nextLine().trim();
    return s.isEmpty() ? defaultVal : Integer.parseInt(s);
}

private static double readDouble(Scanner sc, String prompt
, double defaultVal) {
350     System.out.print(prompt);
    String s = sc.nextLine().trim();
    return s.isEmpty() ? defaultVal : Double.parseDouble(s
    );
}

355 @FunctionalInterface
private interface ThrowingSupplier<T> { T get() throws
    Exception; }

// --- Модель строки результата ---

360 private static class ResultRow {
    final String dataset;
    final int tests, faults;
    final double randomApfd, randomApfdc;

```

```
365     final double addApfd, addApfdc;
        final double gaBaseApfd, gaBaseApfdc;
        final double gaFpApfd, gaFpApfdc;

        ResultRow(String dataset, int tests, int faults,
370             double randomApfd, double randomApfdc,
                double addApfd, double addApfdc,
                double gaBaseApfd, double gaBaseApfdc,
                double gaFpApfd, double gaFpApfdc) {
            this.dataset = dataset; this.tests = tests; this.
                faults = faults;
            this.randomApfd = randomApfd; this.randomApfdc =
                randomApfdc;
375            this.addApfd = addApfd; this.addApfdc = addApfdc;
            this.gaBaseApfd = gaBaseApfd; this.gaBaseApfdc =
                gaBaseApfdc;
            this.gaFpApfd = gaFpApfd; this.gaFpApfdc =
                gaFpApfdc;
        }
    }
380 }
```